Who Wrote It? Identifying LLMs from Their Responses

CMPSC 448 Midterm Project

Team leader: Laasya <last name> | Members: <none or names>

Code: <GitHub repo URL>

1. Problem Definition and Dataset

The goal is to tell which LLM family wrote a response. Given a prompt and a response, I train a classifier to predict the family. I ask two main questions. RQ1: can the response text alone identify the LLM? RQ2: does the prompt help, compared with the response only or the prompt only? I also did RQ3 (does it generalize to unseen task types) and RQ4 (what features separate the models).

Data source. I used the public LMSYS Chatbot Arena Conversations dataset from Hugging Face (lmsys/chatbot_arena_conversations, about 33K conversations). In that dataset, a real user prompt is answered by two anonymous models, so every conversation gives two labeled (LLM, input, output) examples. I kept English conversations only, used the first turn (first user prompt and first reply), and dropped replies shorter than 5 words. I did not collect any private data.

LLM families. I grouped the raw model names into four families: **gpt** (gpt-4, gpt-3.5-turbo), **claude** (claude-v1, claude-instant-v1 and related), **palm** (palm-2), and **llama** (Llama and Llama-derived chat models such as Vicuna, Koala, Alpaca). After filtering there were 33,642 examples, but the classes were unbalanced (llama 17,748; gpt 7,671; claude 5,600; palm 2,623). To balance them I randomly sampled 1,000 examples per family, giving 4,000 total.

Splits. Stratified 80/10/10 split with a fixed seed: 3,200 train, 400 validation, 400 test (100 per family in the test set, so chance accuracy is 25%). For RQ3, I labeled each prompt as coding, math, writing, or qa with a simple keyword heuristic. The 4,000 sampled examples are 2,887 qa, 458 writing, 409 coding, 246 math.

2. Models and Training

Preprocessing. Lowercase, split into word and punctuation tokens with a regex, vocabulary of the 20,000 most common training tokens (plus PAD, UNK, SEP). Responses are truncated to 256 tokens and prompts to 64. For the input+output setting I concatenate prompt, a SEP token, and response into one sequence.

CNN. Embedding (128 dim), three parallel 1D convolutions with kernel sizes 3, 4, 5 and 100 filters each, ReLU, max-pooling over time, concatenation (300 features), dropout 0.5, then a linear layer to 4 classes.

RNN. Embedding (128 dim), one bidirectional LSTM layer (hidden size 128 per direction), masked mean pooling over the non-padding positions, dropout 0.5, then a linear layer to 4 classes.

Training. Adam, learning rate 1e-3, weight decay 1e-5, batch size 64, 12 epochs, cross-entropy loss. I keep the weights from the epoch with the best validation accuracy and report test accuracy and macro F1 from that checkpoint. Every model uses the same seed (0). Models were trained on a CPU (MacBook Air).

3. Results (RQ1 and RQ2)

Test accuracy and macro F1 (4 classes, chance = 25%, n = 400 test examples).

Input to classifier	CNN acc	CNN F1	BiLSTM acc	BiLSTM F1
Output only (RQ1)	63.3%	0.627	54.3%	0.526
Input (prompt) only	27.8%	0.269	27.0%	0.259
Input + output	61.5%	0.616	56.8%	0.543

RQ1. The response text alone is enough to identify the LLM family far above chance. The CNN reached 63.3% and the BiLSTM 54.3%. The CNN was better by about 9 points. With 400 test examples, one accuracy has roughly +/- 2.5 points of sampling noise, so I only treat gaps of about 5 points or more as meaningful.

RQ2. The prompt by itself is close to chance (27.8% and 27.0%). Adding the prompt to the response did not clearly help: the CNN went down by 1.8 points and the BiLSTM went up by 2.5, both within noise. This makes sense for this dataset. In Chatbot Arena, the same prompt is sent to two different models, so a prompt carries almost no information about which model answered. The small non-chance signal in input-only may come from prompt topics or styles that differ a bit between the model groups. The prompts are not a useful fingerprint here, but the response style is.

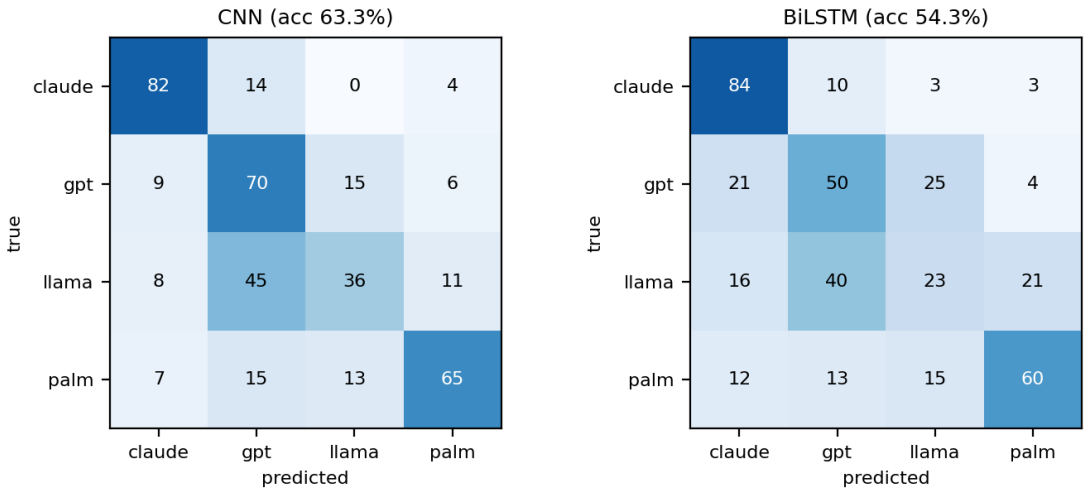


Figure 1. Confusion matrices on the test set, response only (rows = true class, 100 per class).

Per-class recall for the CNN: claude 82%, gpt 70%, palm 65%, llama 36%. The weakest class is llama, and 45 of its 100 test examples were predicted as gpt. One possible reason (I did not test it) is that the llama group is mostly Llama-derived chat models fine-tuned on ChatGPT-style data, so their writing looks like gpt. Claude is the easiest to identify.

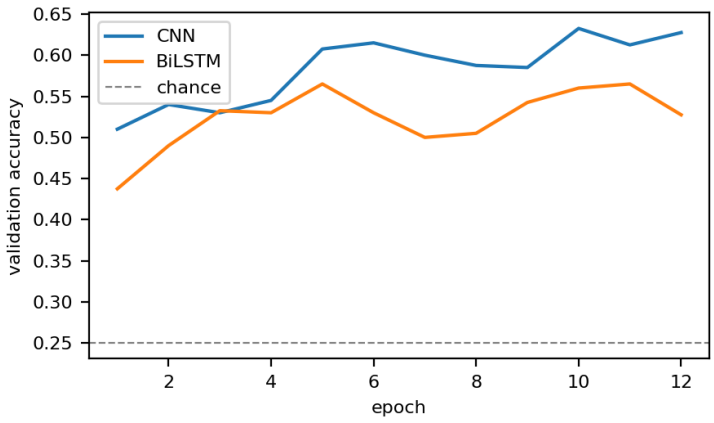


Figure 2. Validation accuracy per epoch, response only. The BiLSTM peaks early and fluctuates, while its training loss keeps dropping, which points to overfitting.

4. In-Depth Analyses

4.1 RQ3: generalization across tasks

I trained on three task types and tested on the fourth (response only). A small share of training data was used for validation. The in-distribution baseline from Section 3 is 63.3% (CNN) and 54.3% (BiLSTM).

Held-out task	Test n	CNN acc	BiLSTM acc
coding	409	57.5%	45.2%
math	246	55.3%	48.0%
qa	2887	50.3%	43.7%
writing	458	57.9%	56.1%

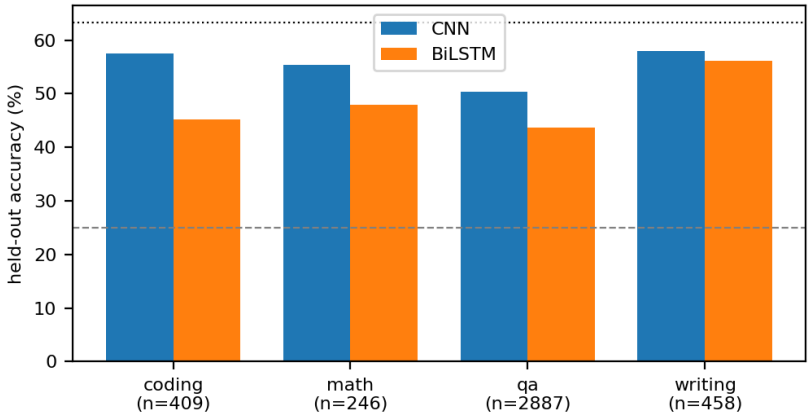


Figure 3. Held-out task accuracy. Dotted line = in-distribution CNN baseline, dashed = chance.

Accuracy drops on unseen tasks, but it stays well above 25%, so part of the fingerprint carries across tasks and part is task specific. The CNN dropped from 63.3% to 57.5% on coding, 55.3% on math, 57.9% on writing, and 50.3% on qa. Writing transferred best, and the BiLSTM barely changed there (56.1%). QA looks like the worst case, but it is confounded: holding out qa removes 2,887 of 4,000 examples, so that model trained on only about 1,100 examples, and the drop may partly come from less data and not only from the task shift. The math and coding sets are small (246 and 409), so their numbers are noisy. The task labels also come from a keyword heuristic, so some prompts are mislabeled.

4.2 RQ4: what distinguishes the models

Average response statistics per family (on the balanced 4,000 examples):

Feature	claude	gpt	llama	palm
Characters	1023	852	735	976
Words	168.0	135.8	121.8	169.0
Type-token ratio	0.659	0.705	0.661	0.595
Newlines	14.8	10.1	6.6	12.0
Bold marks (**)	0.006	0.000	0.017	1.394
Bullet lines	1.03	0.42	0.34	1.94
Numbered lines	1.45	1.82	1.26	0.65
Exclamation marks	0.52	0.25	0.18	0.22
Avg word length	5.00	5.02	5.01	4.64

The families differ in clear ways. Claude writes the longest and most exclamatory answers. Llama writes the shortest answers with the fewest line breaks. Palm is the formatting outlier: it uses markdown bold far more than the others (about 1.4 per response versus near 0), more bullets, and has the lowest lexical diversity and shortest average word. GPT has the highest type-token ratio and uses the most numbered lists.

To see how much of the signal is structural, I ran a logistic regression on only these 13 handcrafted features and two ablations on the neural models:

Setting (response only)	CNN acc	BiLSTM acc
Baseline (full text)	63.3%	54.3%
Formatting stripped (markdown, code blocks removed)	62.8%	59.5%
Truncated to first 60 words	59.5%	55.3%
13 handcrafted features + logistic regression	44.8%	n/a

Length and formatting features alone reach 44.8%, well above chance, so the signal is partly structural (the largest logistic regression weights were for word count, bold marks, and character count). But the neural models reach 63%, so they use wording as well. Stripping formatting did not hurt the neural models (the BiLSTM even improved by about 5 points, which is probably noise or a side effect of cleaner input), so they do not mainly depend on markdown. Cutting responses to 60 words cost the CNN about 4 points, which suggests length and later parts of a response help a little but are not the main signal. Overall, the signals seem to be both linguistic and structural, with wording being the bigger part. I did not run a multi-seed test, so these ablation differences are suggestive and not conclusive.

5. Lessons and Experience

More data mattered a lot. My quick test with 300 examples per family gave about 51% for the CNN and results that jumped around. With 1,000 per family the same model reached 63%. The small run also made ablation comparisons unreliable, so I learned to check the size of the test set before reading anything into a few points of difference.

The CNN beat the LSTM on this data. The BiLSTM's training loss kept falling while its validation accuracy stayed flat or bounced around, so it overfit. The CNN pools the strongest local phrases, which fits a task where short stylistic patterns matter. With more data or pretrained embeddings the gap might shrink.

I expected the prompt to leak the label, and it did not. At first I worried that input-only would be suspiciously good. Looking at how the dataset is built (each prompt goes to two models) explained why it is near chance. I also realized the same prompt can end up in both train and test with different labels, because I

split by example and not by prompt. A split by prompt would be cleaner.

Engineering problems. The newest pandas changed groupby/apply behavior and broke my balancing and statistics code (a tuple key made saving the results to JSON fail). I replaced those parts with plain loops. I also had to log in to Hugging Face to download the data.

Limitations. Only one random seed and one split, so there are no confidence intervals. The data comes from 2023-era models. The llama group mixes several derived models, and gpt-4 and gpt-3.5 are merged, so families are fuzzy. Task labels are heuristic. Only first-turn conversations were used.

What I would do next. Repeat with several seeds, split by prompt, add newer models and more families, try pretrained embeddings or a transformer baseline, and test whether the classifier still works when responses are paraphrased or length-normalized.

AI assistance. I used Claude to help write and debug the code and to help interpret results. I ran all experiments myself and the numbers above come from my own runs.